SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Duration :60 Mins

On Class QUESTIONS: 5

Total Marks:50

Annexure A

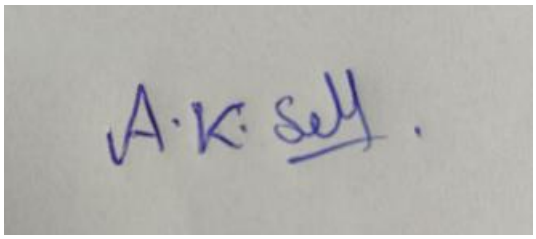
INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I A K Selva Prabhu (192421224) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action

Signature of the student:



Q1. Smart Electricity Billing System

(i) Suitable declarations

```
int units;
float rate;
float tax_rate;
float energy_cost;
float tax_amount;
float total_bill;
```

• units represents the number of electricity units consumed.

• rate represents the cost per unit.

• tax_rate represents the applicable tax percentage/rate.

• energy_cost, tax_amount, and total_bill are declared as float because monetary calculations may contain decimal values.

(ii) Three-Address Code

Given:

```
energy_cost = units × rate
tax_amount = energy_cost × tax_rate
total_bill = energy_cost + tax_amount
```

Three-address intermediate code:

```
t1 = units * rate
energy_cost = t1

t2 = energy_cost * tax_rate
tax_amount = t2

t3 = energy_cost + tax_amount
total_bill = t3
```

Optimized form:

```
t1 = units * rate
t2 = t1 * tax_rate
t3 = t1 + t2
```

(iii) Quadruples

No.	Operator	Argument 1	Argument 2	Result
1	*	units	rate	t1
2	*	t1	tax_rate	t2
3	+	t1	t2	t3

Where:

```
t1 = energy cost
t2 = tax amount
t3 = total bill
```

Triples

No.	Operator	Argument 1	Argument 2
0	*	units	rate
1	*	(0)	tax_rate
2	+	(0)	(1)

Here (0) refers to the result of triple 0 and (1) refers to the result of triple 1.

Indirect Triples

Pointer	Points to Triple
P0	0
P1	1
P2	2

```
Triple table:
0:  units * rate
1:  (0) * tax_rate
2:  (0) + (1)
```

The pointer table allows the execution order of triples to be changed without modifying the triple statements.

(iv) Common Subexpressions and Optimization Opportunities

There is no repeated common subexpression such as:

```
a*b
a*b
```

in the given calculation.

The expression:

```
units * rate
```

is calculated only once and its result is reused for calculating both tax and total bill.

Optimization opportunities:

• Avoid unnecessary temporary variables.

• Store units * rate in one temporary variable.

• Reuse the computed energy cost.

• Avoid recomputing units * rate.

• Remove unnecessary assignments to energy_cost and tax_amount if they are not required elsewhere.

(v) Optimized Intermediate Code

```
t1 = units * rate
t2 = t1 * tax_rate
t3 = t1 + t2
total_bill = t3
```

Further optimized if only total_bill is required:

```
t1 = units * rate
t2 = t1 * tax_rate
total_bill = t1 + t2
```

Justification: The optimized code calculates units * rate only once, reuses the result for tax calculation and total calculation, and eliminates unnecessary intermediate assignments. Therefore, the number of operations and temporary variables is reduced.

Q2. Employee Promotion Evaluation System

The employee is eligible when:

```
Experience >= 5 AND (Performance Score > 90 OR Certification = TRUE)
```

(i) Boolean Expression

```
E = (Experience >= 5) AND
    ((Performance_Score > 90) OR (Certification == TRUE))

E1 = Experience >= 5
E2 = Performance_Score > 90
E3 = Certification == TRUE

E = E1 AND (E2 OR E3)
```

(ii) Short-Circuit Intermediate Code

```
if Experience >= 5 goto L2
goto Lfalse

L2:
if Performance_Score > 90 goto Ltrue
goto L3

L3:
if Certification == TRUE goto Ltrue
goto Lfalse

Ltrue:
Eligible = TRUE
goto Lend

Lfalse:
Eligible = FALSE

Lend:
```

(iii) Backpatching

Initially, the jump targets are unknown.

```
1. if Experience >= 5 goto _
2. goto _

3. if Performance_Score > 90 goto _
4. goto _

5. if Certification == TRUE goto _
6. goto _
```

Backpatch the jumps:

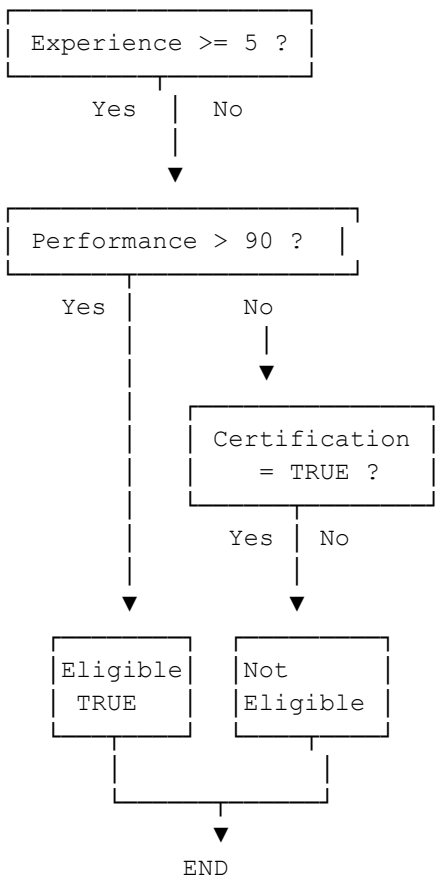
```
1. if Experience >= 5 goto L2
2. goto Lfalse

3. if Performance_Score > 90 goto Ltrue
4. goto L3

5. if Certification == TRUE goto Ltrue
6. goto Lfalse
```

```
E1.truelist  → L2
E1.falselist → Lfalse

E2.truelist  → Ltrue
E2.falselist → L3

E3.truelist  → Ltrue
E3.falselist → Lfalse
```

(iv) Final Labelled Intermediate Code

```
L1:
if Experience >= 5 goto L2
goto Lfalse

L2:
if Performance_Score > 90 goto Ltrue
goto L3

L3:
if Certification == TRUE goto Ltrue
goto Lfalse

Ltrue:
Eligible = TRUE
goto Lend

Lfalse:
Eligible = FALSE

Lend:
```

(v) Control Flow Graph



The short-circuit approach avoids unnecessary evaluation and therefore improves execution efficiency.

Q3. Hospital Patient Priority System

The given priority values are:

```
1 → CRITICAL
2 → HIGH
3 → MEDIUM
4 → LOW
```

(i) Three-Address Code Using Conditional Jumps

```
if priority == 1 goto L1
if priority == 2 goto L2
if priority == 3 goto L3
if priority == 4 goto L4
goto Ldefault

L1:
CRITICAL
goto Lend

L2:
HIGH
goto Lend
```

```
L3:
MEDIUM
goto Lend

L4:
LOW
goto Lend

Ldefault:
INVALID PRIORITY

Lend:
```

(ii) Equivalent Jump-Table Implementation

```
if priority < 1 goto Ldefault
if priority > 4 goto Ldefault

goto JumpTable[priority]

JumpTable:
1 → L1
2 → L2
3 → L3
4 → L4

L1:
CRITICAL
goto Lend

L2:
HIGH
goto Lend

L3:
MEDIUM
goto Lend

L4:
LOW
goto Lend

Ldefault:
INVALID PRIORITY

Lend:
```

(iii) Quadruple Representation

No.	Operator	Arg1	Arg2	Result
1	==	priority	1	L1
2	==	priority	2	L2
3	==	priority	3	L3
4	==	priority	4	L4

5	goto	Ldefault	-	-
6	CRITICAL	-	-	-
7	goto	Lend	-	-
8	HIGH	-	-	-
9	goto	Lend	-	-
10	MEDIUM	-	-	-
11	goto	Lend	-	-
12	LOW	-	-	-
13	goto	Lend	-	-

(iv) Conditional Branching vs Jump Table

Conditional Branching	**Jump Table**
Performs comparisons one by one	Uses an indexed table
Execution time depends on the selected case	Directly reaches the required case
Simple to implement	Requires additional table storage
Suitable for a small or sparse set of values	Suitable for dense consecutive values
May require several comparisons	Provides near constant-time dispatch

(v) Recommendation

For an emergency hospital, the jump-table implementation is suitable because priority values 1 to 4 are consecutive and dense.

• Faster case selection.

• Reduces the number of sequential comparisons.

• Provides predictable execution.

• Suitable for real-time emergency processing.

• Makes case dispatch efficient.

• The four priority values can be directly mapped to their corresponding actions.

• The system can quickly identify critical patients.

Therefore, a jump table is recommended for this particular priority system.

Q4. E-Commerce Payment Processing System

Given procedures:

```
amount = calculateAmount(cart)
processPayment(amount)
sendConfirmation(amount)
```

(i) Intermediate Code

```
param cart
t1 = call calculateAmount, 1
```

```
amount = t1

param amount
call processPayment, 1

param amount
call sendConfirmation, 1
```

Here:

```
t1 = return value from calculateAmount()
amount = calculated payment amount
```

(ii) Parameter Passing

The parameter cart is passed to calculateAmount().

```
param cart
t1 = call calculateAmount, 1
```

The returned value is stored in amount.

Then amount is passed to processPayment():

```
param amount
call processPayment, 1
```

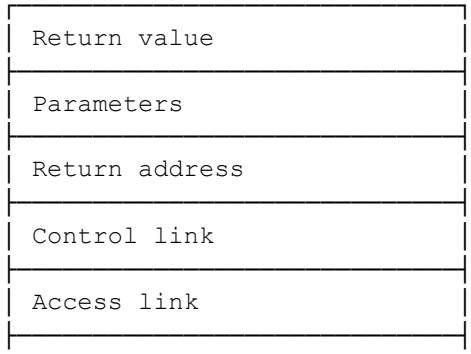
Finally, amount is passed to sendConfirmation():

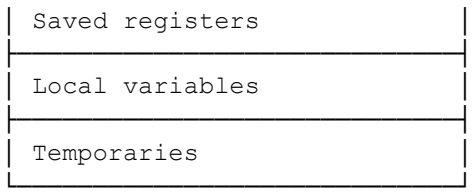
```
param amount
call sendConfirmation, 1
```

Flow:

```
cart
  ↓
calculateAmount(cart)
  ↓
amount
  ↓
processPayment(amount)
  ↓
sendConfirmation(amount)
```

(iii) Activation Record



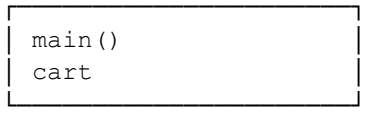


For calculateAmount(cart):

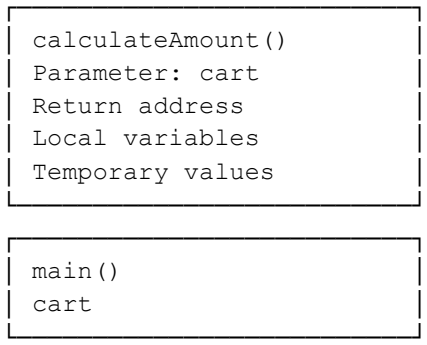
```
Activation Record
-------------------------
Return value
Parameter: cart
Return address
Control link
Saved registers
Local variables
Temporaries
-------------------------
```

(iv) Stack Diagram for Call and Return

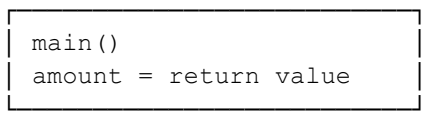
Initial state:



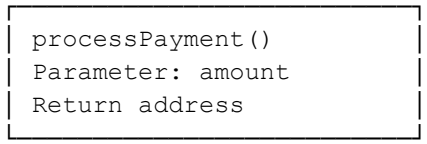
After calling calculateAmount(cart):

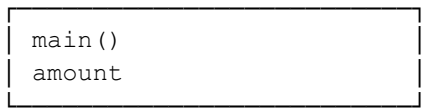


After return:

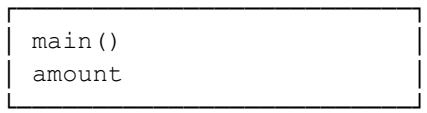


Calling processPayment(amount):





After return:



(v) Procedure Call Management

The caller evaluates the required arguments.

Parameters are placed in the appropriate locations.

The return address is saved.

A new activation record is created.

Control is transferred to the called procedure.

The procedure executes using its parameters and local variables.

The return value is stored when applicable.

The activation record is removed after completion.

Control returns to the saved return address.

The caller continues execution.

Thus, the stack provides an organized mechanism for managing nested procedure calls, parameters, local variables, return addresses, and temporary values.

Q5. Smart ATM Withdrawal Verification System

The customer can withdraw cash if:

```
Account Balance >= Withdrawal Amount
AND
(PIN Verified = TRUE OR Biometric Verified = TRUE)
```

(i) Boolean Expression

```
E1 = Account_Balance >= Withdrawal_Amount
E2 = PIN_Verified == TRUE
E3 = Biometric_Verified == TRUE

E = E1 AND (E2 OR E3)
```

Complete Boolean expression:

```
(Account_Balance >= Withdrawal_Amount)
AND
((PIN_Verified == TRUE) OR (Biometric_Verified == TRUE))
```

(ii) Short-Circuit Intermediate Code

```
if Account_Balance >= Withdrawal_Amount goto L2
goto Lfalse

L2:
if PIN_Verified == TRUE goto Ltrue
goto L3

L3:
if Biometric_Verified == TRUE goto Ltrue
goto Lfalse

Ltrue:
Withdraw = TRUE
goto Lend

Lfalse:
Withdraw = FALSE

Lend:
```

Short-circuit behavior:

```
If balance is insufficient
        ↓
Immediately reject withdrawal

If balance is sufficient
        ↓
Check PIN

If PIN is verified
        ↓
Allow withdrawal

Otherwise
        ↓
Check biometric verification

If biometric is verified
        ↓
Allow withdrawal

Otherwise
        ↓
Reject withdrawal
```

(iii) Backpatching

Initial code with unresolved jumps:

```
1. if Account_Balance >= Withdrawal_Amount goto _
2. goto _

3. if PIN_Verified == TRUE goto _
4. goto _

5. if Biometric_Verified == TRUE goto _
6. goto _
```

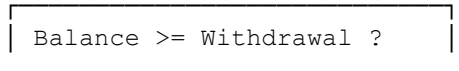
After backpatching:

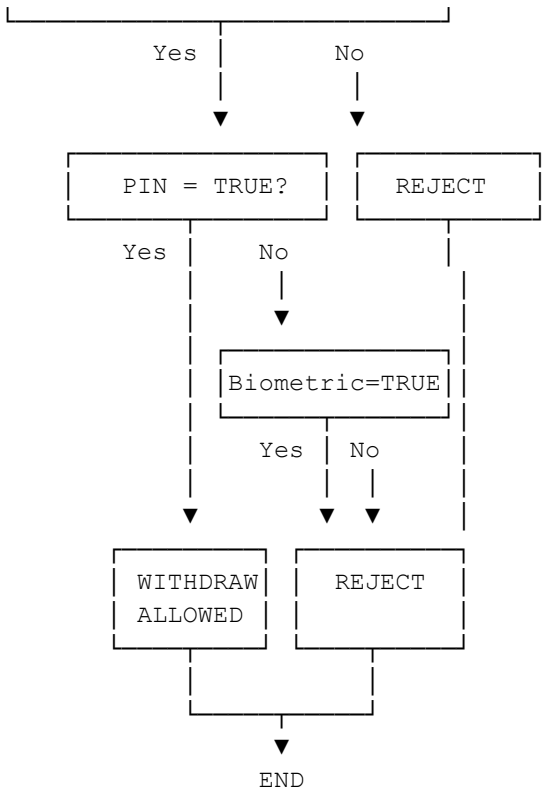
```
1. if Account_Balance >= Withdrawal_Amount goto L2
2. goto Lfalse

3. if PIN_Verified == TRUE goto Ltrue
4. goto L3

5. if Biometric_Verified == TRUE goto Ltrue
6. goto Lfalse

E1.truelist  → L2
E1.falselist → Lfalse

E2.truelist  → Ltrue
E2.falselist → L3

E3.truelist  → Ltrue
E3.falselist → Lfalse
```

(iv) Final Labelled Intermediate Code

```
L1:
if Account_Balance >= Withdrawal_Amount goto L2
goto Lfalse

L2:
if PIN_Verified == TRUE goto Ltrue
goto L3

L3:
if Biometric_Verified == TRUE goto Ltrue
goto Lfalse

Ltrue:
Withdraw = TRUE
goto Lend

Lfalse:
Withdraw = FALSE

Lend:
```

(v) Control Flow Graph





The short-circuit implementation improves efficiency because biometric verification is performed only when the account has sufficient balance and PIN verification has failed.